Slack Bytes in Structures : Explained with Example

geeksforgeeks.org/slack-bytes-in-structures-explained-with-example

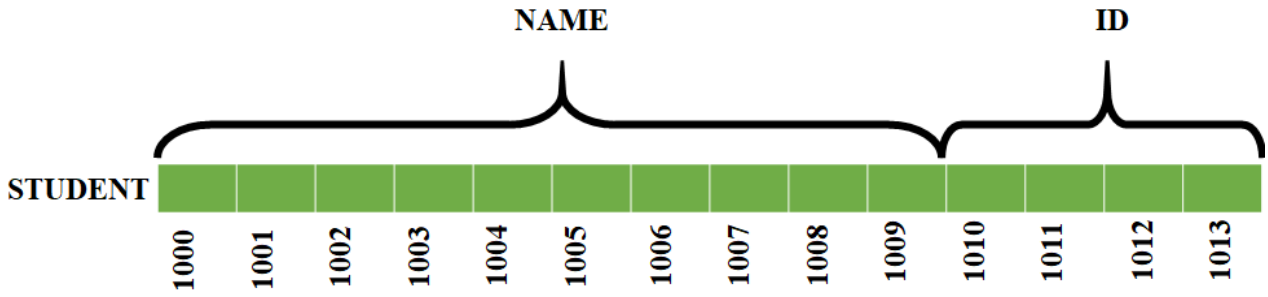
December 3, 2019

- Difficulty Level : Medium
- Last Updated : 16 Aug, 2021

Structures:
Structures are used to store the data belonging to different data types under the same variable name. An example of a structure is as shown below:

```
struct STUDENT
{
    char[10] name;
    int id;
};
```

The **memory space for the above structure** would be allocated as shown below:



Here we see that there is no empty spaces in between the members of the structure. But in some cases **empty spaces** occur between the members of the structure in and these are known as ***slack bytes***.

Need for Slack Bytes?
In computers and technology ***speed*** is a very important factor. We always try to make designs and algorithms in such a way that the access to the memory is much faster or the solution is much faster to obtain. The microprocessors access the data that is present in the even addresses faster than that of the data that is present in the odd addresses. So, we try to allocate the memory of the data in such a way the starting address of the particular variable is in the even address.

Microprocessors access the data that is stored in the even address faster than the data that is stored in the odd address. So, it becomes the responsibility of the compilers to allocate the memory for the members such that they have even addresses so that the data can be

accessed very fast. This leads to the concept of slack bytes.

SLACK BYTES:

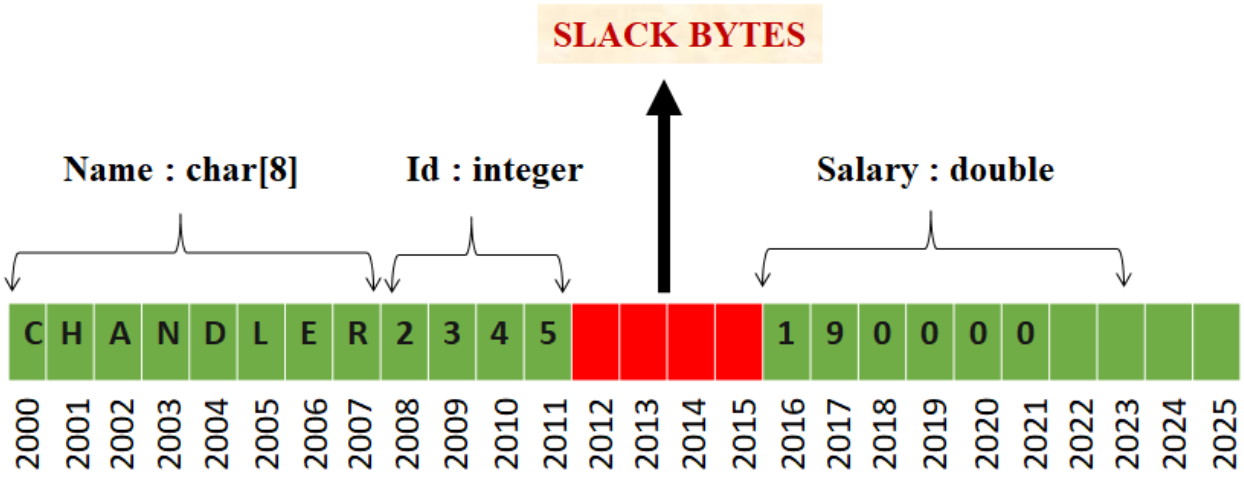
The optimized compilers will always assign even addresses to the members of the structure so that the data can be accessed very fast. The even addresses may be multiples of 2, 4, 8 or 16. This introduces some unused bytes or holes between the boundaries of some members. These ***unused bytes or holes between boundaries of the members of structure are known as slack bytes***.

The slack bytes do not contain any useful information and they are actually a wastage of memory space. But the access of the data would be much faster in the case of slack bytes concept. When the slack bytes are used, the size of the structure would be greater or the same as the sum of the size of the individual members of the structure. Some optimized compilers assign the address such that the addresses of each member will be multiple of the size of the largest data type of the structure.

Example: Consider the following declaration of a structure:

```
struct EMPLOYEE
{
    char name[8];
    int id;
    double salary;
};
```

Here we see that the double datatype occupies the highest memory space (***assume double = 8 bytes***). Hence every member would be assigned to a memory address which are a multiple of 8 as shown below:



Observe here that all the members of the structure start with a address which is a multiple of 8. And we can observe an empty space in between (region shaded in red) which is known as the slack bytes.

Want to learn from the best curated videos and practice problems, check out the **C Foundation Course** for Basic to Advanced C.